Overview

DocChat-AI is a document question answering application. A user uploads PDF or TXT files and then asks questions in plain English. The system finds the most relevant passages and uses a large language model to write an answer that is based only on those passages. Every answer includes citations showing the file name and page number.

The project was built as a learning exercise in Retrieval-Augmented Generation, usually shortened to RAG. It is written entirely in Python, uses Streamlit for the user interface, ChromaDB as the vector database, and the Google Gemini API for both embeddings and answer generation.

Goals

The first goal is accuracy: the assistant must never invent facts that are not in the uploaded documents. The second goal is transparency: users can always open the source passages behind an answer. The third goal is simplicity: the whole pipeline is written without LangChain so that every step can be understood and explained.

Supported Files

The application accepts PDF and TXT files. Scanned PDFs without a text layer are not supported in version one. The maximum recommended file size is 20 megabytes, and the system has been tested with documents of up to 200 pages.

Document Ingestion

The ingestion pipeline begins when a user uploads a file. The loader reads the file page by page using the pypdf library. For every page it stores the extracted text together with the file name and the page number. Keeping the page number is important because it allows the final answer to cite exactly where the information came from.

Empty pages are skipped. If a file is corrupted or has an unsupported format, the loader raises a clear error message that is shown to the user in the interface. Text files are treated as a single page numbered one.

Cleaning the Text

Extracted text often contains extra spaces, broken line endings, and hyphenated words split across lines. A light cleaning step normalises whitespace but does not remove punctuation, numbers, or capitalisation, because these can carry meaning. Headers and footers that repeat on every page are a known source of noise and are listed as a future improvement.

Metadata

Each piece of text carries three metadata fields: the file name, the page number, and a unique chunk identifier. The identifier follows the pattern filename underscore p page number underscore c chunk number, for example report_p3_c2. These identifiers make it possible to update or delete a single file without touching the rest of the index.

Chunking Strategy

Language models and embedding models work best with focused pieces of text, so each page is divided into smaller chunks. The default chunk size in DocChat-AI is 800 characters, with an overlap of 100 characters between neighbouring chunks.

Naive chunking cuts text at fixed positions and often splits words or sentences in half. DocChat-AI avoids this by looking for the nearest paragraph break or sentence ending before the size limit is reached, and cutting there instead.

Why Overlap Matters

An answer can sit on the boundary between two chunks. Without overlap, the first half of a sentence might end up in one chunk and the second half in another, and neither chunk would match the question well. The overlap repeats the last 100 characters of one chunk at the start of the next, so important sentences stay intact in at least one chunk.

Experiments

During testing, chunk sizes of 400, 800, and 1200 characters were compared on a set of 30 questions. Chunks of 400 characters retrieved precise passages but often lacked enough context for a complete answer. Chunks of 1200 characters contained more context but diluted the match score with unrelated sentences. The size of 800 characters gave the best balance and answered 26 of the 30 questions correctly, which is an accuracy of about 87 percent.

Embeddings and Vector Storage

An embedding is a list of numbers that represents the meaning of a piece of text. Texts with similar meanings produce vectors that point in similar directions. DocChat-AI uses the Gemini embedding model to convert every chunk into a vector. Chunks are embedded in batches of 50 to reduce the number of API calls and to stay within rate limits.

The vectors are stored in ChromaDB, an open source vector database that runs locally and saves its data in a folder named chroma_db. The collection is called docchat and uses cosine distance as its similarity measure. Cosine similarity compares the angle between two vectors, which works well for text.

Upserts

When a file is uploaded again, DocChat-AI uses an upsert operation. Because chunk identifiers are stable, the same chunk is simply overwritten instead of being stored twice. This prevents duplicate results in the search.

Task Types

The Gemini embedding model supports task types. Document chunks are embedded with the retrieval document task type, while user questions are embedded with the retrieval query task type. Using the correct task type improves the quality of the match between a question and its answer passage.

Retrieval and Answering

When a user asks a question, the question is embedded and compared with all stored vectors. The four most similar chunks are retrieved. This number is called top k and is set to 4 by default.

Each retrieved chunk has a similarity score. Chunks with a score below 0.45 are discarded as irrelevant. If no chunk passes this threshold, the system replies that it could not find the answer in the uploaded documents, and the language model is not called at all. This behaviour prevents the assistant from guessing.

Prompt Construction

The prompt sent to Gemini contains a system instruction, the numbered context chunks with their file names and page numbers, a short history of recent conversation turns for follow-up questions, and finally the user question. The system instruction tells the model to answer only from the context, to be concise, and to cite sources in square brackets.

Streaming

Answers are streamed to the screen word by word so that users see a response immediately instead of waiting for the full text. The source passages are displayed below the answer in a collapsible section.

Deployment

DocChat-AI is deployed on Streamlit Community Cloud. The Gemini API key is stored in the platform's secrets manager and is never committed to the repository. The requirements file lists all dependencies so the environment can be rebuilt with a single install command.

Evaluation

A script named run_eval.py reads 30 question and answer pairs from a JSON file, runs each question through the pipeline, and checks whether the expected key fact appears in the generated answer. It prints the overall accuracy and lists the questions that failed so they can be investigated.

Known Limitations

The system cannot read text inside images, tables are flattened into plain text, and very long documents take a few minutes to index the first time. Answers that require combining information from many different pages may be incomplete because only four chunks are retrieved.

Future Work

Planned improvements include optical character recognition for scanned files, a re-ranking step to improve the order of retrieved chunks, support for Word documents, a chat with CSV mode for spreadsheets, and user accounts so that each person has a private document library.